spi-nor驱动

全志V3S嵌入式驱动开发（spi-nor驱动）

原创

嵌入式-老费 已于 2023-08-29 15:15:40 修改

阅读量1.3k 收藏 2

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文介绍了如何在全志V3S Licheepi Zero平台上进行SPI NOR驱动开发，包括修改设备树文件、内核代码以适配MX25L25645G SPI NOR芯片，编译内核并烧录，以及验证驱动是否成功加载。通过这些步骤，可以实现脱离SD卡束缚，降低成本的全SPI NOR系统。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

所谓的spi nor驱动，其实就是让spi nor芯片，在linux kernel 5.2.y启动后可以被正确地识别到。前面有一篇文章，我们说到了spi nand驱动，其实它和spi nor是一样的。我们同样地也需要对设备树和内核做一点修改，其他部分几乎不需要动。这样，结合之前的uboot spi改造结果，就可以完完全全构建一个适配spi nor的full image，彻底脱离开sd卡的束缚。毕竟，从成本上考虑，spi nor还是便宜很多。

注意，这里使用的spi nor还是mxic公司的MX25L25645G。

1、修改sun8i-v3s-licheepi-zero.dts文件

之前的spi0是为适配spi nand服务的，所以这里要修改成适配spi nor，最典型的修改就是compatible变成了“jedec,spi-nor”，之前是“spi-nand”，

```
&spi0 {
        status ="okay";
        mx25l25645g:mx25l25645g@0 {
                compatible = "jedec,spi-nor";
                reg = <0x0>;
                spi-max-frequency = <50000000>;
                #address-cells = <1>;
                #size-cells = <1>;
        };
};
```

2、修改内核代码

在kernel 5.2.y当中，有两处内核代码需要修改。一个是drivers/mtd/devices/m25p80.c文件，在m25p_ids中添加一个新的芯片名称，和dts中的芯片保持一致；

```
{"mx25l25645g"},
```

另外一个就是在drivers/mtd/spi-nor/spi-nor.c，在spi_nor_ids添加一条新的注册信息，

```
{ "mx25l25645g", INFO(0xc22019, 0, 64 * 1024, 512, 0) },
```

其中mx25l25645g表示nor flash型号，0xc22019表示RDID的内容，*641024表示sector的数量，512表示每一个sector中byte的个数。所以总的大小就是641024*512=32M byte，256M bit，两者不冲突。*

有的同学也许会问，这个0xc22019应该从哪里获取，其实可以先找到mx25l25645g的芯片手册，比如通过搜索引擎查找下，

```
https://www.tme.eu/Document/b050a55b550e3061ddd7611588a5a8b0/MX25L25645G,%203V,%20256Mb,%20v1.7.pdf
```

然后搜索RDID这个关键字，就可以在文档中找到对应的芯片设备信息了，

Table 6. ID Definitions

Command Type		MX25L25645G		
RDID	9Fh	Manufacturer ID	Memory Type	Memory Density
		C2	20	19
RES	ABh	Electronic ID		
		18		
REMS	90h	Manufacturer ID	Device ID	
		C2	18	
QPIID	AFh	Manufacturer ID	Memory Type	Memory Density
		C2	20	19

CSDN @嵌入式-老费

3、编译内核，烧入dtb和zImage文件

编译内核的办法就是make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-。生成的dtb文件和zImage文件则通过读卡器直接烧入到sd卡即可。

4、插入sd卡，检验spi norflash是否已经被linux kernel发现

因为目前还没有制作完整的、适合norflash的image，所以我们还是通过sd卡来进行测试。经过uboot的加载，可以借助于打印日志看到kernel和rootfs已经被正确加载，此时通过检测是否存在/dev/mtd*节点，就可以判断出spi norflash设备是否被正确加载了。这和spi nandflash的检测过程是一样的，节点名称也是一样的。

```
# ls -l /dev/mtd*
crw-------    1 root     root       90,   0 Jan  1 00:00 /dev/mtd0
crw-------    1 root     root       90,   1 Jan  1 00:00 /dev/mtd0ro
```